

Reviewer Pack — Colin de Verdière Invariant Lower Bounds Resolution Length f...

Entry 23bccf812fdb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 19:29:16 UTC

Colin de Verdière Invariant Lower Bounds Resolution Length for Tseitin Formulas

Entry ID: 23bccf812fdb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 19:29:16 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Colin de Verdière Invariant) **Field B** (complexity object): Resolution Proof Length for Tseitin Formulas

Statement:

For any Tseitin formula derived from a graph G , the resolution proof length is $\geq 2^{\{\Omega(\mu(G))\}}$, where $\mu(G)$ is the Colin de Verdière invariant of G . If G is a non-expander, $\mu(G) \leq O(1)$.

Rationale (proposer's reasoning):

The Colin de Verdière invariant captures topological obstructions to graph embeddings, which may directly influence the structural complexity of resolution proofs for Tseitin formulas. Its polynomial-time computability for small graphs enables direct testing.

Taxonomy category: TSEITIN_RES (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bb1d4a74b7b10d02

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        factor = Fraction(A[i][i])
        for k in range(i+1, n):
            A[k][i] /= factor

    # Back substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i+1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= Fraction(A[i][i])

    return x

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def eigenvalues(A):
    n = len(A)
```

```

if n == 1:
    return [A[0][0]]

# Characteristic polynomial coefficients
coeffs = [1]
for i in range(n):
    coeffs.append(-sum(A[i][:i+1]))
    for j in range(i):
        coeffs[j] -= A[i][j] * coeffs[j+1]

# Find roots using Newton's method
def f(x):
    return sum(coeffs[k] * x**k for k in range(n+1))

def df(x):
    return sum(k * coeffs[k] * x**(k-1) for k in range(1, n+2))

roots = []
for _ in range(n):
    x0 = random.uniform(-10, 10)
    while True:
        fx = f(x0)
        dfx = df(x0)
        if abs(dfx) < 1e-6:
            break
        x0 -= fx / dfx
    roots.append(x0)

return sorted(roots)

def colin_de_verdiere_invariant(G):
    n = len(G)
    A = [[Fraction(G[i][j]) for j in range(n)] for i in range(n)]
    A_inv = gaussian_elimination(A)
    eigenvals = eigenvalues(A_inv)
    mu_G = max(eigenvals) - min(eigenvals)
    return float(mu_G)

def tseitin_formula(G):
    n = len(G)
    clauses = []
    for v in range(n):
        clauses.append([v+1])
        for u in range(v):
            if G[u][v] == 1:
                clauses.append([-u-1, -v-1, v+1])
                clauses.append([-u-1, v+1, -v-1])
    return clauses

def dpll(clauses, assignment=[]):
    if not clauses:
        return True
    unit_clauses = [c[0] for c in clauses if len(c) == 1]
    pure_symbols = {}
    for c in clauses:
        for lit in c:

```

```

        if abs(lit) not in pure_symbols:
            pure_symbols[abs(lit)] = lit > 0
        elif pure_symbols[abs(lit)] != (lit > 0):
            return False

    if unit_clauses or pure_symbols:
        new_assignment = assignment[:]
        for lit in unit_clauses + list(pure_symbols.keys()):
            new_assignment.append(lit)
        return dpll(clauses, new_assignment)

    literal = next(lit for lit in range(1, len(clauses)+1) if lit not in [abs(a) for a in assignment])
    if dpll([c[:] for c in clauses if literal not in c and -literal not in c], assignment + [literal]):
        return True
    if dpll([c[:] for c in clauses if -literal not in c], assignment + [-literal]):
        return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0

    mu_G = colin_de_verdiere_invariant(G)
    if math.isinf(mu_G) or math.isnan(mu_G):
        return {
            "metric_name": "resolution_length",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    clauses = tseitin_formula(G)
    resolution_length = len(clauses) if dpll(clauses) else float('inf')

    return {
        "metric_name": "resolution_length",
        "metric_value": resolution_length,
        "instances_tested": 1,
        "conjecture_holds": resolution_length >= 2 ** mu_G,
        "counterexample": "" if resolution_length >= 2 ** mu_G else f"resolution_length={resolution_length}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70bd8319.py", line 162, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70bd8319.py", line 136, in run_trial
    mu_G = colin_de_verdiere_invariant(G)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70bd8319.py", line 87, in colin_de_verd
    A_inv = gaussian_elimination(A)
           ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70bd8319.py", line 31, in gaussian_elim
    A[k][i] /= factor

```

```

File "/usr/lib/python3.12/fractions.py", line 615, in forward
    return monomorphic_operator(a, b)
           ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/fractions.py", line 763, in _div
    raise ZeroDivisionError('Fraction(%s, 0)' % db)

```

ZeroDivisionError: Fraction(1, 0)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the division-by-zero error in Gaussian elimination for Fraction objects and rerun tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	73737
2	propose	ollama_remote	qwen3:8b	0	0	62457
3	propose	ollama_remote	qwen3:8b	0	0	107802
4	preregistration	ollama_remote	qwen3:8b	0	0	24077
5	novelty	ollama_remote	qwen3:8b	0	0	20849
6	novelty	ollama_remote	qwen3:8b	0	0	17457
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13211
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17175
9	judge	ollama_remote	qwen3:8b	0	0	18354

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 355119 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/23bccf812fdb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/23bccf812fdb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/23bccf812fdb.tar.gz` (if generated)